# CS 4122: Reinforcement Learning

## Programming Assignment #1 (for Module 1)

Release date: 15th March, 2025, 9:30 am (IST)

Due date: 8th March, 2025, 11:59 pm (IST)

Maximum Score: 100 marks (this assignment is graded)

Read the following before you move forward:

1. This programming assignment is worth 10% of the total marks of this course.

2. Late policy: You can be late by a maximum of 3 days. Your assignment will not be accepted more than 3 days after the due date. The actual score and received score (received score is actual score minus penalty for late submission) are related as follows:

$$\text{received score} = \begin{cases} \text{actual score} & ;\text{late by 0 days} \\ 0.9 \cdot \text{actual score} & ;\text{late by 1 day} \\ 0.8 \cdot \text{actual score} & ;\text{late by 2 days} \\ 0.7 \cdot \text{actual score} & ;\text{late by 3 days} \\ 0 & ;\text{late by more than 3 days} \end{cases}$$

3. This is a team project. You have to do the project with the team that was finally selected for you. Only one submission per team. To submit, go to your team's Google drive folder. The Google drive link will be emailed to you soon. Then go to the sub-folder titled Programming Assignment 1 and drop the following FOUR files (DON'T zip/compress these files, DON'T submit any other files):
   ***(a) report.pdf*** ***(b) lin_greedy.py*** ***(c) lin_ucb.py*** ***(d) policy_gradient.py***
   You must read section 8 carefully to understand what needs to be included in these four files.

4. The first page of report.pdf should contain a detailed description of the contribution made my individual team members. NOTE: Documenting the work is NOT a valid contribution.

5. Even though each team members may contribute to a specific section of the assignment, every team member should have the technical understanding of the entirety of the project.

6. I will be conducting vivas. Assignment marks will be heavily dependent on these vivas.

7. Plagiarism, if detected, will lead to a score of ZERO for all the team members!
   a. It is ok to take help from AI tools. It is down right stupid to copy-paste from AI tools and think that is acceptable.
   b. It is ok to take help from other teams. But, it is a bad idea to keep open another teams work in front of you while you do the assignment. While you may think you are just taking help, your reports and your codes may get heavily influenced by the other team to the point that it will qualify as being plagiarized.

# Dynamic Pricing in Ride Sharing using Contextual Bandits

## Section 1: Objectives

The following are the main objectives of this programming assignment:

1. Demonstrate a real-life application of contextual bandits that you learned in module 1.
2. Hands-on training to use OpenAI Gymnasium.
3. Implement various algorithms for contextual bandits that you learned in module 1 from scratch.
4. Demonstrate how understanding the working of bandit algorithms can help you in designing bandit algorithms for specialized bandit settings.

## Section 2: Installation

1. I assume that you already have a setup to code in Python. Typically, this would mean that you have Anaconda installed in your system. Anaconda has both Spyder and Jupyter Notebook that can be used to code in Python. NOTE: Even if you are using Jupyter notebook for coding, the final codes that you submit must be Python scripts (.py files) as mentioned in the 1st page of this document**.** Else, you get ZERO for all those questions where coding is involved.
2. If you are using Anaconda, execute the following pip commands in Anaconda prompt to install the libraries required for this assignment (for Colab, *pip* should be replaced with *!pip*):
   *pip install gymnasium*
   *pip install swig*
   Please Note: You need to install OpenAI Gymnasium and NOT OpenAI Gym *(Gym and Gymnasium are two different libraries that contains environments for reinforcement learning. Gymnasium is the newer version of Gym and supports more environments than Gym. Hence, install Gymnasium)*.
3. You also have to install a Deep Learning library like Keras, Tensorflow, or Pytorch. This will be required to implement policy gradient algorithm.

## Section 3: Sharing Workload

The way I see it there are four tasks. **Task 1:** Finding efficient algorithm to compute shortest Manhattan distance between a source and a destination in a 2D occupancy map. **Task 2:** Implementing $\varepsilon$-greedy and UCB algorithm for linear bandits. **Task 3:** Implementing policy gradient algorithm. **Task 4:** One theory question.

If there are four members, four of you can divide these four tasks among yourself. If there are three members, the one who is doing task 1 can do task 4 as well. Now, tasks 2 and 3 (especially task 2) is dependent on task 1. But still, it is possibly to do both in parallel (check problem 1 of section 8 for details).

## Section 4: Useful Resources

1. All the required pseudocodes are there in lectures 7 to 11.
2. The following video lecture might be helpful:

   https://youtu.be/LfH0DzDBtgs

   NOTE:
   - The first 36 minutes of this video is about how to use OpenAI Gymnasium environment. Other resources to learn this topic are:
     - The video lecture of lecture 0, part 3. The first 52 minutes is enough.
     - The following YouTube video by Nicholas Renotte. The first 8 minutes are enough. https://www.youtube.com/watch?v=cO5g5qLrLSo&t=581s&ab_channel=NicholasRenotte
   - The last 50 minutes is this video is about how to implement $\varepsilon$-greedy policy for linear bandits in Python. Note that this video is from Fall 2024. The update rule that I use for $\varepsilon$-greedy policy may look different from that in the slides but nothing that you don't know.
3. You also need to know deep learning and how to code in either Keras, Tensorflow, or Pytorch. In the beginning of the course, I emailed you a bunch of resources in this regard. You can also refer to lecture 0, part 2 where I teach you how to use Keras.

## Section 5: Problem Setup

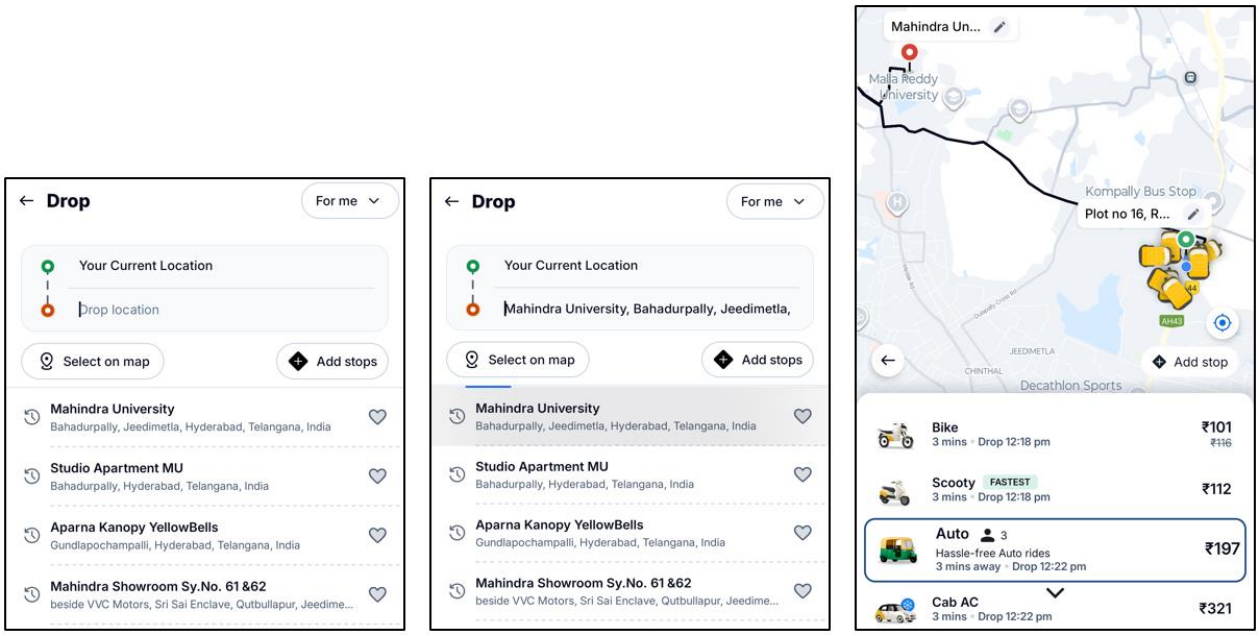


*Figure 1*

Figure 1 shows the first two steps when we use a ride sharing app. *First,* you choose your drop-off location (pickup location is generally your current location). *Second,* the app shows you the price and estimated time of arrival (ETA) of the driver for various ride options like bike, auto, cab etc. In this programming assignment, we simplify this problem by making the following assumptions:

- The app allows only one ride option (say auto).

- Only the price is shown. ETA of the driver is NOT shown.
- The app serves ride request on first-come-first-served basis. The app does not wait for multiple passengers to request rides and then show the price according to the current market demand.
- The app does NOT do long-term planning when deciding what price to show, i.e. it is not a reinforcement learning problem. One such long term plan is to purposefully show a high price so that the passengers do not book the cab. This maybe done if the drivers are limited and the trip length is short (short length trip means less money). The app may plan to reserve the limited number of drivers for the future hoping longer length trip. We ignore such long-term planning.

This problem can be formulated as a contextual bandit where the ride sharing app is the agent. The price shown to the passenger is the action and the information about the passenger and the available drivers are the context. Reward may be the commission fee earned by the ride sharing company, or the money earned by the driver, or the number of successful rider-passenger mapping (success here would imply both the passenger and the driver accepted the ride).

**Supervised learning OR contextual bandit?**

A problem can be solved using multiple approaches. It is a good engineering practice to ask which of these approaches is better suited for the problem. For e.x., one way to solve this problem is to train a regression model whose features are the information about the passengers and the drivers and whose label is the price to show to the passenger. Since supervised learning is so successful, it is important to ask the question: "Why to use contextual bandits to solve this problem?" Here are a few reasons why contextual bandit is a better approach to solve this problem as compared to supervised learning:

1. We need labels to train a regression model. In this case, the price is the label. However, there is no guarantee that the data contains the best price to show to the passenger, e.x. maybe the passenger would have accepted the ride even if the price was higher than the target variable. This is phenomenon is called partial feedback and was discussed during lecture. Contextual bandits are better suited for setups with partial feedback.
2. What price to show to the passenger highly depends on the map of the area. Map of an area changes with time. A supervised learning model trained on past data may not give optimal results for the current map. Contextual bandits are better suited for such setups because the data is collected online.

## Section 6: The Gym Environment

You are given a Python file *RideSharing.py* that has a class *DynamicPricingEnv*. This class implements an OpenAI Gymnasium environment to simulate the generation of contexts and reward for the problem setup mentioned in section 5. In this section, we explain the necessary details of *DynamicPricingEnv* that is required to solve this programming assignment. You don't have to understand the code in *RideSharing.py* other than what is explained in this section.

Any environment for a RL/bandit is associated with the concept of episode. An episode is a sequence of consecutive time slots associated with a certain task. An episode starts from time slot $t = 0$ and continues

for a maximum of say $H$ time slots. After $t = H - 1$, the episode ends. After an episode ends, we have to reset the environment in order to start a new episode back from time slot $t = 0$. In our setup, a time slot is of $5$ seconds duration, and an episode is of one hour, i.e. $H = 60 \cdot 12 = 720$.

*Figure 2: 2D map of the area under consideration*

We assume that in every time slot, one and only one passenger requests a ride using the app. A passengers' request in time slot $t$ is associated with the following information:

1. It's current location, $(x_t^o, y_t^o)$, and its destination location, $(x_t^f, y_t^f)$, from the map shown in figure 2.
2. An estimate of the passenger's price sensitivity, $\alpha_t^p$. A passenger's price sensitivity is a measure of how much the passenger is willing to pay per unit distance. Higher the price sensitivity, more is the passenger willing to pay per unit distance.

There are $n_t$ drivers who is near to the passenger in time slot $t$. $n_t$ varies with time slot. It is a random integer between $1$ and $10$. We assume that there is at least one nearby driver, i.e. $n_t$ is at least $1$. The $k^{th}$ driver is associated with the following information:

1. It's current location, $(x_{k,t}^d, y_{k,t}^d)$, from the map shown in figure 2.
2. An estimate of the driver's price sensitivity, $\alpha_{k,t}^d$. A driver's price sensitivity is a measure of how much the driver wants to get paid per unit distance. Higher the price sensitivity, more the driver wants to get paid per unit distance.

**CONTEXT:** The context, $c_t$, for time slot $t$ is a tuple $c_t = (c_t^p, c_t^d)$ where:

1. $c_t^p$ contains passenger information. It is a Numpy array of the form $c_t^p = [x_t^o, y_t^o, x_t^f, y_t^f, \alpha_t^p]$.
2. $c_t^d$ contains driver information. It is a tuple of Numpy arrays of the form,
$$c_t^d = ([x_{1,t}^d, y_{1,t}^d, \alpha_{1,t}^d], [x_{2,t}^d, y_{2,t}^d, \alpha_{2,t}^d], \cdots, [x_{n_t,t}^d, y_{n_t,t}^d, \alpha_{n_t,t}^d])$$
Since $n_t$ varies with time slot, the number of Numpy arrays in $c_t^d$ varies with time slot.

You can run the following code to get a better understanding of the context.

```
from RideSharing import DynamicPricingEnv

env = DynamicPricingEnv()

x, _ = env.reset()      # Run this line multiple times
```

The variable $x$ is the context. You can look into the components of variable $x$ to get a better understanding of the context. Also, you should run the third line multiple times to get an idea of $n_t$ varies with time slot.

ACTION: The action in time slot $t$, $a_t$, is the price that the ride sharing app (which is the agent here) decides to show to the passenger and the drivers. $a_t$ is a scalar. Without any loss of generality, we assume that $a_t$ is between $0$ to $1$, i.e. it is a normalized price. The drivers and the passenger are both shown the price $a_t$ that is decided by the app; there is no additional strategic decision that is made by the app to decide what price to show to individual drivers. There is an understanding between the drivers and the app that if $a_t$ is the price shown, then the driver will earn $(1-\theta)a_t$ and the ridesharing company will earn $\theta a_t$ as the commission fee. $\theta \in (0,1)$ is the commission ratio.

REWARD: After $a_t$ is decided, it is first shown to the passenger. If the passenger does not go ahead with the booking, reward in time slot $t$ is zero. If the passenger accepts the price, then the $n_t$ nearby drivers are approached one after another. The drivers are approached in the ascending order of their distance from the passenger until one of them accepts the ride request. If none of the drivers accepts the ride, then the reward in time slot $t$ is again zero. If at least one of the drivers accepts the ride, then the reward in time slot $t$ is the commission fee $\theta a_t$ that is earned by the company.

## Section 7: Using the Gym Environment

Again, you don't have to understand the code in *RideSharing.py*. You just have to know how to use a Gymnasium environment. Section 4 lists a few resources which might be helpful in this regard.

**What you are not allowed to do!**

As you know, contextual bandit is a learning setup. So, while coding the agent/policy, you are not supposed to know the internal working of *DynamicPricingEnv* other than those topics mentioned in the sections 5 and 6. Now, I can't stop you from looking into the code of *DynamicPricingEnv* and understanding it. But, if I find that any part of your implementation used some internal working of *DynamicPricingEnv* that you are not supposed to know, then it will negatively affect your score. To be on the safe side:

1. You can use any intuition that you drew from section 5 and 6.
2. You can use only the attributes/functions of *DynamicPricingEnv* that is mentioned immediately after this when implementing the agents/policies.

Say that you create and instance of the class *DynamicPricingEnv* as follows: *env = DynamicPricingEnv()*. Then, you can use the following attributes/functions of *env* when implementing the policies:

1. env.Horizon. This is $H$; the total number of time slots in an episode. In our case, it is $720$.
2. env.MaxDrivers. This is the maximum number of nearby drivers that is available in any time slot. For this setup, it is $10$.
3. env.MaxRideCost. This is the maximum price that we can charge the passenger. For this setup, it is $1$.
4. env.commission_ratio. This is commission ratio. For this setup, it is $0.2$.
5. env.observation_space. The is the context space associated with this environment.
6. env.action_space. This is the action space associated with this environment.

7. env.reset(). The tutorials on Gymnasium listed in section 4 explains how this function works.
8. env.step(action). The tutorials on Gymnasium listed in section 4 explains how this function works.
9. Other than the attributes of *env*, you call also you the map of the area that is there in *map_agent.png*. You MUST NOT use *map_environment.png*. *map_environment.png* is the true map of the area while *map_agent.png* is the map of the area according to the ride sharing company which may or may not be as accurate as the true map.

## Section 8: The Deliverables

**Setting the right expectation**

Before we get started with the deliverables, you should know that the best average reward that you can get for this environment is around 0.025. Also, if you choose price uniformly at random, you will get an average reward of 0.013. My personal best is 0.023. These three numbers should help you decide how good your agent in performing.

1. (10 marks) Recall that linear bandits may need some feature engineering to make it work. Based on your reading of sections 5 and 6, find useful features from the context that will help linear bandits perform better. You may also try to train linear bandits in the next two problems and decide that they are performing well without any feature engineering.
   Other than finding useful features, you should also address how you are planning to handle varying context size (because number of nearby drivers is varying) when training the agents in the subsequent sections. The answer to these questions must be given ONLY in **report.pdf**.
   - One of the useful features is Manhattan distance between passenger's location and destination (maybe distance between passenger and the drivers is also important). To do so, you need to run a shortest path algorithm on *map_agent.png*. You are free to use any shortest path algorithm that you want but keep in mind that with high probability, the shortest path algorithm that you choose is likely to be the most time-consuming part of the policy. You are free to take inspiration from the shortest-path algorithm that I have used in *RideSharing.py*. Your answer to this question should include what shortest path algorithm you are using (possibly with a pseudocode but not necessary).
   - **A word of wisdom:** It may take some time to figure of the shortest path algorithm. One of the team members should do this while others get busy with the remaining problems. Afterall, you may find out that shortest path is not that useful a feature.

2. (15 marks) In the blank Python script *lin_greedy.py* that is provided to you, implement an agent for the *DynamicPricingEnv* environment that uses ε-greedy algorithm for linear bandit. This Python script should also plot a receding window time-averaged reward incurred by the agent. You MUST NOT include anything in **report.pdf** for this part other than the plot of the receding window time averaged reward.
   - ε-greedy algorithm for linear bandit needs discrete action space. But here, the action space is continuous. You can discretize the continuous action space by binning the actions. How many bins you use is upto you. The number of parameters of the model increases with number of

bins making it harder to train. But, if the number of bins is too small, you will end up choosing sub-optimal actions.

- What is receding window time averaged reward? Check this small 9 minutes video. Window size of $2000$ time slots in recommended.
- The number of episodes you train and how you vary your exploration rate is upto you.

3. (15 marks) In the blank Python script *lin_ucb.py* that is provided to you, implement an agent for the *DynamicPricingEnv* environment that uses UCB algorithm for linear bandit. This Python script should also plot a receding window time-averaged reward incurred by the agent. You MUST NOT include anything in **report.pdf** for this part other than the plot of the receding window time averaged reward.
   - Same instructions as lin-greedy.

4. (30 marks) In the blank Python script *policy_gradient.py* that is provided to you, implement an agent for the *DynamicPricingEnv* environment that uses policy gradient algorithm for contextual bandits. This Python script should also plot a receding window time-averaged reward incurred by the RL agent. You MUST NOT include anything in **report.pdf** for this part other than the plot of the receding window time averaged reward.
   - You may choose to discretize the action space but you may not get good results. I may deduct 5-10 points if you discretize the action space.
   - If you choose to use continuous action space, then be careful when computing the loss during gradient descent *(minus of the expected reward is the loss and then you do gradient descent to maximize the expected reward)*. You have to add a "not so obvious term" when computing the loss. This term arises because of the use of sigmoid function to limit the actions between $0$ to $1$. I may deduct 5 points if you don't include this term.
   - Your policy gradient algorithm may not need shortest distance as one of the feature. In fact, there are pros of cons of including the same. The pro is that you can use simpler neural network if shortest distance information is included. Otherwise, you need a more complicated neural network to implicitly learn the shortest distance information. There are two cons. *First,* the time required to compute the distance. But this may not exactly be a con because if you don't use shortest information then you need to use a more complicated neural network that takes longer to train. *Second,* if *map_agent.png* substantially differs from the true map, then shortest distance information will be erroneous. My "free advice" will be to not use shortest distance information for policy gradient. Just FYI, my neural network has at most four layers.
   - **A word of wisdom:** Most of the Deep-RL algorithms (policy gradient is one of them) takes some amount of training before it starts showing performance. So initially, you may not see any progress and you may want to stop the training. Avoid this temptation! It took be around 40-75 episodes to see any visible increase in receding average reward during training.
   - Using baselines might help to "stabilize training".

5. (10 marks) It is always a good idea to test whether the policies that you trained is giving logical output. It is possible to do so even though you don't know the internal working of the environment. Based on the explanation in sections 5 and 6, you should have some intuition of how the price predicted by your agent should vary depending on the context. In this question, you are required to present **two tests**

that demonstrates if your agent is performing as desired. What test you do is upto you. The answer must contain two components. *First,* the details of what test you did. *Second,* the result of the test in form of graphs and/or tables. The answer to this part should be there in **report.pdf** only. Don't submit any code for this part; not even in **report.pdf**.

6. This question is not related with the previous five questions. The answer to this question should be there in **report.pdf** only.

If you see the lecture slides on policy gradient, you will notice that I very conveniently ignored that action space varies with the context. In this question, we address this issue for discrete action space. Let $\mathcal{A}(x)$ be the action space for context $x$. We know that $\mathcal{A}(x)$ is a subset of $\{1, 2, \cdots, A\}$ for all $x$ where $A$ is the total number of actions across all contexts. Let's define a function $m(a, x)$ as follows,

$$m(a,x) = \begin{cases} 1\,, a \in \mathcal{A}(x) \\ 0\,, a \notin \mathcal{A}(x) \end{cases}$$

Let $\pi_\theta(a|x)$ be a policy that is defined over the action space $\{1, 2, \cdots, A\}$ that does not take it account that action space varies with context. Answer the following questions:

**(a)** Write expression for a policy $\tilde{\pi}_\theta(a|x)$ in terms of $\pi_\theta(a|x)$ and $m(a, x)$ that has zero probability of taking actions that is not in the action space.

**(b)** Let $J(\theta)$ be the expected reward per time slot of policy $\tilde{\pi}_\theta(a|x)$. Give an expression to estimate the gradient of $J(\theta)$ with respect to $\theta$. You must simplify the expression so that it is in terms of $\pi_\theta(a|x)$, $m(a, x)$, and $\frac{\partial \pi}{\partial \theta}$.